

Diskless Viewstamped Replication with Unbounded Crash-Stop Reincarnation

— Addendum: experimental results

Simon Massey

Abstract—This document is an addendum to *Diskless Viewstamped Replication with Unbounded Crash-Stop Reincarnation*; the parent paper’s abstract and keywords are not repeated here. This companion reports only measured and checked findings, each with its observed numbers, its evidence path in the repository and the command that reproduces it. It adds no method and changes nothing in the parent paper or in the proof-ladder addendum; where the repository records no measurement, the experiment’s contract is stated and its findings are marked *to be measured*.

I. SCOPE OF THIS REPORT

Every entry below is a result the repository evidence records: a model-check log, a kernel-checked proof transcript, a checker’s summarised counts, a dated audit, or a merge record. No experiment was executed for this report, and no number is inferred, extrapolated or invented. Each finding carries its provenance path and, in the appendix, the command that reproduces it.

The evidence classes are: TLC model checking of the era and reincarnation design models (Section II); mutation and negative controls, in TLA+, in the Lean mutation harness and inside the proof-ladder rungs (Section III); the Showboat-verified rung transcripts 13–28 (Section IV); the independent exhaustive checker’s counts for weighted reachability and the fused batch (Section V); the dated Lean axiom audits (Section VI); and the stress bench, the Zig twin and the Maelstrom harness (Section VII).

One intended measurement has no recorded numbers anywhere in the repository and is marked *to be measured* throughout: the stress-bench lane figures beyond those preserved in the merge record (Section VII). This report stands alone; the parent paper [1] supplies the method and the design, and the proof-ladder addendum [2] supplies the mathematical statements that the transcripts below check.

II. MODEL CHECKING

The TLA+ models are `formal/VrrCoreEras.tla` (the era-transition design model) and `formal/VrrCoreReincarnation.tla` (the Crash-Stop-Self-Evict forced sequence). The archived campaign ran TLC 2.19 (rev. 5a47802) from a jar pinned by SHA-256 in the toolchain manifest (936a2620...), breadth-first with two workers on a 2GB heap. Each run’s wall limit, worker

count and elapsed time are recorded beside its log as a JSON digest in `evidence/tlc/` within the Lean project. Table I quotes the archived green runs; Table II the expected rejections. All logs and digests live under `formal/uvrr-lean/evidence/tlc/`.

The reincarnation model’s invariant set is recorded in `formal/README.md`: `FrownChain` (every committed quorum is a strict majority and consecutive committed eras overlap), `EvictedNeverVoter`, `MassRule` (at most mass one per node per committed era), `NoStandbyVote`, `RebornAfterFence`, `SequenceCompletes`, `LeaderIsVoter`, and the liveness property `Completes`. The stall witness is the intentionally failing run: with three deaths, two live voters hold weight 2 of 4, no strict majority exists, and TLC reports `Completes` violated by a trace that ends stuttering in the stalled state — the recorded existence proof that the model stalls rather than committing without a majority.

Two discrepancies between records are noted rather than hidden. First, `formal/README.md` records `VrrCoreEras.cfg` as “837,204 distinct / 3,358,487 generated, depth 32, 3m24s”, while the archived log records 837,204 distinct / 2,743,933 generated, depth 32, 137.7s: the distinct-state counts and depth agree, the generated totals and the generated totals and elapsed times differ between the two records, and this report quotes the archived log. Second, `formal/README.md` additionally records an era-model campaign whose raw logs are not archived under `evidence/tlc/`: the even-split four-node exhaustive run (green, 290,474 distinct / 1,571,856 generated, depth 30), the fixed-seed simulation runs (10,000 depth-40 crash traces, 900,119 checked states; and 100,000 depth-60 traces, seed 1, 26,793,084 checked states, 1 h56 m), the two intentionally false witness invariants (reached in 207 and 125 distinct states), the no-pivot-guard experiment (green, 837,204 distinct states), and the era mutations M2, M4, M5 (`CommittedLogsAgree`) and M7, M8 (refused by the named startup gates, zero states). Their provenance is `formal/README.md` and their reproduction commands are the `make tla` family in the appendix.

III. MUTATION CONTROLS

Three independent control suites are recorded; each rejection is a checked result, not a review observation.

TABLE I

ARCHIVED TLC 2.19 RUNS, GREEN; THE .CFG SUFFIX IS IMPLIED. STATES ARE REPORTED AS DISTINCT / GENERATED. ELAPSED TIMES ARE THE WRAPPER’S RECORDED WALL TIME.

Configuration	What was checked	Distinct	Generated	Depth	Elapsed
VrrCoreEras	Era model, three nodes, exhaustive; no crash	837,204	2,743,933	32	137.7 s
VrrCoreReincarnation2	Two-node forced sequence; b crashes, reincarnates as d	35	162	11	0.76 s
VrrCoreReincarnation3	Three-node forced sequence, fixed leader	28	221	14	0.72 s
VrrCoreReincarnationDouble	Three nodes at double weight (all weights 2)	28	221	14	0.71 s
VrrCoreReincarnationFive	Five-node forced sequence, fixed leader; f reincarnates as d	3,406	53,078	20	1.07 s
VrrCoreReincarnationCrash	Five nodes, leader crash forced between the two eras, one view change, two deaths	1,253,444	26,308,632	31	108.5 s
VrrCoreReincarnationFiveViews	Five nodes, one live-leader replacement plus unrationed dead-leader replacement, all invariants	17,394,566	371,140,753	32	27 m 35 s
VrrCoreReincarnationFiveCompletes	Liveness: Completes under FairSpec, leader may die and be replaced	126,301	2,318,131	26	19.0 s

TABLE II

ARCHIVED TLC 2.19 RUNS, EXPECTED REJECTIONS AND THE STALL WITNESS; THE .CFG SUFFIX IS IMPLIED. MUTATION RUNS USE ONE WORKER; THE VIOLATED INVARIANT IS THE CHECKED REJECTION. STATES AT THE COUNTEREXAMPLE ARE REPORTED AS DISTINCT / GENERATED.

Configuration	Defect checked	Violated	States
VrrCoreErasM1	Era 1 drops the transferred tail entry while reporting its frontier	FrontiersOrdered	2,111 / 5,425
VrrCoreReincarnationM1	Era 1 skips the decrement; era 2 leaves the victim at weight 2	MassRule	9 / 26
VrrCoreReincarnationM2	One-era swap: the unfenced victim evicted and the joiner promoted in one era	MassRule	11 / 22
VrrCoreReincarnationM3	The sequence runs without the fence (no bump)	RebornAfterFence	38 / 160
VrrCoreReincarnationM4	The sequence aborts after era 1; era 2 never proposed	SequenceCompletes	8 / 19
VrrCoreReincarnationM5	Weight-0 identities’ replies counted in quorums	NoStandbyVote	9 / 31
VrrCoreReincarnationFiveM1	The M1 defect at five nodes	MassRule	61 / 268
VrrCoreReincarnationFiveM2	The M2 defect at five nodes	MassRule	29 / 81
VrrCoreReincarnationFiveStall	Intentionally failing witness: three deaths leave two live voters with weight 2 of 4	Completes	59,666 / 426,526

A. The Lean mutation harness

formal/uvrr-lean/check_mutations.py compiles temporary mutants and requires Lean to reject each. Its recorded design and outcome (the proof-ladder addendum’s appendix states the suite’s five rejections and the hole detection): an existential quorum checker (Q2.all replaced by Q2.any); an unsafe four-node decision family (the 3-of-4 subsets replaced by [[3]]); an omitted next-slot guard in NormalLog; an append after the view-change fence in ViewFence; and a cross-view prepare receipt in LogProvenance. The suite additionally checks that a sorry is detected through sorryAx in the axiom query even when the compiler exits zero. Reproduction: python3 check_mutations.py from formal/uvrr-lean.

B. The TLC defect regressions

The nine expected rejections of Table II (one era-model mutant, five three-node mutants, two five-node mutants, one stall witness) each enable exactly one weakened perimeter through the Defect constant and must exit nonzero on the named invariant; the configuration-to-obligation mapping is recorded in formal/README.md, and the make tla-mutations lane requires both the failure and the exact violated-invariant text. The era-model mutations M2, M4, M5, M7 and M8 are recorded in formal/README.md as above.

C. Negative controls inside the rungs

The rung transcripts record their own controls alongside the verified results: rung 13’s length-only selector loses a newer committed prefix and an omitted scalar length bound misranks reports; rung 14’s authentic out-of-order delivery witnesses failure when the slot guard is bypassed; rung 15’s concrete post-fence append breaks the historical report; rung 16’s delayed wrong-view prepare control checks the model interface; rung 23 states the honest three-node degenerate negative (the forming quorums coincide, so no pivot exists and none is exercised); rung 24’s casting-vote verdict splits by boundary (degenerate across E0/E1, genuine pivot across E1/E2); rung 26’s premature promotion of the replacement (claiming weight one in the intermediate era) is rejected by Lean with an application type mismatch; and rung 28’s handwritten swap_control kernel-checks that the equal-total swap $(1, 2, 1, 2) \rightarrow (2, 1, 2, 1)$ makes $\{B, D\}$ a majority at E_0 and a minority at E_1 — the preservation hypothesis of the telescoping theorem is not free. Rung 27’s transcript additionally records that both Leanstrat generation rounds for Fuse.lean failed and that the accepted proof bodies were repaired locally: the successful result is the repaired file, not an unassisted generator success.

IV. PROOF-LADDER RUNGS

Table III records what each Showboat-verified rung transcript actually checked. The transcripts live in `formal/uvrr-lean/ladder/<NN>-<name>.md`; the numbering runs 13–16 and 22–28 (no transcripts 17–21 exist). Each transcript embeds its Showboat identifier, the module source and the observed check output; the replay command is `showboat verify ladder/<NN>-<name>.md` from `formal/uvrr-lean` after `lake build`.

One tooling control is recorded beside the rungs (the proof-ladder addendum’s appendix): `tests/test\_leanstral\_driver.py` has two offline regressions. A mock API forces two concurrent proof loops to finish writing their candidates before either copies its result, exposing the shared-file race deterministically, and a second assertion checks that the prompt preserves project imports. Both failed on the original driver and pass with per-invocation scratch directories and an import-preserving prompt.

V. EXHAUSTIVE CHECKER COVERAGE

The independent Python checker (`research/weighted-reachability/check.py`, standard library only) records its exact coverage counts in `research/weighted-reachability/generated/summary.json`, and the repository’s written record states the headline totals (`research/weighted-reachability/README.md`).

A. Weighted reachability

Every weight vector and availability count with a positive live leader through ten existing identities: 310,007 symmetry-representative profiles, covering 14,541,815 labelled weight/availability profiles, with zero failed checks. The maximum phantom weight added is 3 (matching the proved bound $\delta \leq 3$), and the maximum hub path is 19 unit edits. Every source/target pair satisfying the premises through six identities: 774,159 representative pairs, covering 3,939,756 ordered labelled pairs, zero failed checks, every constructed path shortest in unit edits (maximum path 11). The per-identity breakdown is preserved in `generated/summary.json` and rendered in `generated/report.html`. The same summary records 250 independent cross-checks of the power-set intersection rule, and separate negative controls that reject an unsafe two-vote jump, a missing live majority, excessive phantom weight, and counting the abstract companion as a received quorum.

B. The fused batch

`research/weighted-reachability/check\_fuse.py` records (`generated/fuse-summary.json`): the three-voter fused schedule checks 2 boundaries and 64 cross-quorum majority pairs; the five-voter schedule checks 6 boundaries and 5,760 pairs; over four identities with weights in $\{0, 1, 2\}$, all 29,690 common-response endpoint pairs admit a monotone unit-edit path (99,352 unit edges) that keeps a responding majority at every step and

is shortest; and the negative control walks the equal-total swap $(1, 2, 1, 2) \rightarrow (2, 1, 2, 1)$ through unit edits, recording responder masses 4, 4, 3, 3, 2 and quorum flags true, true, false, false, false — the responding quorum is not preserved along a legal unit-step path.

Reproduction for both checkers is in the appendix. These finite counts are computational evidence; the general claims rest on the Lean proofs and the mathematical note recorded in the same directory.

VI. AXIOM AUDITS

`formal/uvrr-lean/check\_axioms.py` queries every named theorem, definition and abbreviation in the compiled project and permits only `propext`, `Quot.sound` and `Classical.choice`. Three dated audit outcomes are recorded in the repository:

- 1) The rung 26 transcript (11 September 2026): the build completed 24 jobs and the audit passed 355 declarations; direct queries record that `ReincarnationSafety`, `five_safe` and `ReincarnationAgreement`, `five_leader_overlap` depend only on the three standard axioms.
- 2) The proof-ladder addendum’s appendix: the extended build completes 25 jobs and the audit covers 412 declarations.
- 3) The rung 28 transcript (14 September 2026, recorded against the pre-addendum source): 27 Lake jobs, the audit covered 450 declarations with only the permitted axioms; `Fuse.Telescope.telescope_pass` and `Fuse.Telescope.swap_control` both report the three standard axioms.

No audit count is recorded at this addendum’s HEAD; the three counts above are the dated records the repository holds. Reproduction: `lake build` then `python3 check_axioms.py` from `formal/uvrr-lean`.

VII. STRESS BENCH, ZIG TWIN AND MAELSTROM HARNESS

A. The Maelstrom stress bench

The bench’s contract is the make lane set in the Makefile: the Maelstrom lin-kv workload against five nodes ($f = 2$), 180-second runs at rate 20 with nemesis interval 10, one lane per fault profile — `test-clean` (no faults), `test-partition`, `test-kill`, and `test-resurrect` (the kill nemesis against persisted nodes, where every kill reopens with no stopped quorum and the boot bumps the identity, a T3 reincarnation). The linearizability verdict is Maelstrom’s own Knossos check; the adapter tests (`tests/maelstrom\_adapter.rs`) assert the node’s end of the transport against a scripted relay, and the recorded note is that the vendored harness is SIGKILL-only, so no clean-stop lane exists — the T1–T2 clean-stop discipline is pinned by the adapter test `tests/maelstrom_adapter.rs case a_clean_shutdown_reopens_under_the_same_identity`.

The recorded lane results are preserved in the merge record of the stress bench’s restoration (commit `b0bf2f9`, 14 September 2026, and its landed twin `44da48e`): “Lanes:

TABLE III
THE RUNG TRANSCRIPTS 13–28: WHAT EACH VERIFIED RUN CHECKED, AS RECORDED IN THE TRANSCRIPT. ALL RESULTS ARE KERNEL-CHECKED BY LEAN 4.33.1 WITHOUT MATHLIB.

Rung	Verified finding as recorded
13	Whole-log selection by last-normal view: the executable selector returns a member of maximal (view, length) rank, and quorum intersection with the stated temporal premises yields committed-prefix preservation; the bounded scalar rank is proved equivalent to the TLA model's.
14	Normal-log message provenance: fixed-view operational induction derives comparable report logs and retention of replica prefixes; receivers enforce the next slot without any global agreement test.
15	Historical whole-log reports across view fences: multi-view local executions derive the selector's voter-history premise; a node raises its floor before reporting and retains immutable vote and reply histories.
16	Shared multi-view committed-log compatibility: the inductive invariant discharges same-view comparability and installed-base extension; strong view induction proves committed-log compatibility for a fixed configuration without crashes.
22	The reincarnation specification fixed as definitions: the durable identity contract, the incarnation bump, the two-era forced weight sequence, the phase machine, the higher-identity-wins read rule and continuation commitment. (The transcript carries a dated note that the governing boot rule is now the marker transition machine; the transcript is left as recorded.)
23	The casting vote instantiated on the reincarnation eras: at two nodes every E0/E1 quorum pair is decisive at the leader and the boundary overlap holds through the casting-vote structure; the three-node case is the recorded degenerate negative.
24	The five-voter reincarnation rung: majority families computed (not axiomatised) at the concrete configurations; era safety by the general one-unit discharge; the casting-vote verdict splits by boundary.
25	Agreement across the forced sequence under every view schedule: <code>sequence_p1</code> derives <code>P1</code> , <code>sequence_agreement</code> instantiates the era-indexed agreement theorem, and the five-node instance carries the E1/E2 leader-overlap pivot.
26	The final composition <code>ReincarnationSafety.five_safe</code> : agreement, replacement weights, eviction from voting authority along the forced run, and identity non-reuse, from the existing history contract. The composition was generator-supplied in two bounded rounds, kernel-checked, then shortened without changing the statement. Observed: build 24 jobs; axiom audit PASS 355 declarations.
27	Fuse: finite response-quorum telescoping: <code>eligible_through</code> , <code>chosen_all</code> , <code>decide_all</code> , and the concrete eligibility calculations for the three-transition three-voter and seven-transition five-voter schedules; first-quorum negative control. Generation failed twice; the accepted bodies were repaired locally.
28	The Telescoping Theorem <code>Fuse.Telescope.telescope_pass</code> : for a three-node schedule with two transitions under the fuse constraints, the first transition passing telescopes the remaining slots. The <code>swap_control</code> negative control is hand-written; the theorem's proof body is generator-drafted and kernel-checked unchanged. Observed at integration: 27 Lake jobs; axiom audit 450 declarations; the Rust corpus green, 33 suites, no inline test modules under <code>src/</code> .

test-clean 0.7407, test-partition 0.2415/0.2710, test-kill 0.5813, test-resurrect 19 dirty T3 bumps — all `:valid? :true.` Every lane's linearizability check passed; the resurrection lane recorded 19 dirty T3 identity bumps across its kills. The decimal figures are recorded in the merge message without further elaboration, and the raw Maelstrom run artefacts are not committed (the store and state directories are gitignored); the merge message is the recorded provenance. Lane figures beyond that record are *to be measured*.

B. The Zig twin

The vendored TigerBeetle 0.17.9 storage stack (`zig/`, every patched line recorded in `zig/PATCH\_MANIFEST.md`) drives the marker transition machine over real direct-IO paths, compiled with the pinned Zig 0.14.1. The recorded outcome (the same merge records): 19 of 19 Zig tests pass with the pinned toolchain, including the exhaustive 4⁴ marker-assignment test over the pure machine noted in `zig/README.md`. The demo consumer is `examples/uvrr-reincarnation/` (feature `uvrr`), which runs the T1/T2/T3 transitions and membership replay over the vendored IO.

C. The Rust corpus gate

The rung 28 transcript records the corpus gate at its date: `cargo test -features maelstrom green, 33 suites, and no inline test modules under src/`. The repository gate (`make check`) adds the format check, `clippy` at `-D warnings`, and the `cli`-featured run.

VIII. REPRODUCTION COMMANDS

From the repository root unless stated. The TLC lanes use the hash-pinned jar recorded in `formal/uvrr-lean/evidence/tlc/toolchain.json`.

```
make check
make test-clean
make test-partition
make test-kill
make test-resurrect

zig test -lc -dep stdx
-Mroot=zig/root.zig
-Mstdx=zig/stdx/stdx.zig

python3
research/weighted-reachability/check.py
-max-n 10 -pair-max-n 6

python3
research/weighted-reachability/check_fuse.py

cd formal/uvrr-lean && lake build
```

```

    && python3 check_axioms.py
python3 check_mutations.py
for f in ladder/[0-9][0-9]-*.md; do
    showboat verify "$f" || exit; done

make tla    # era model, Docker
make tla-mutations
make tla-local TLA2TOOLS_JAR=<tla2tools.jar>
java -XX:+UseParallelGC -Xmx2g
    -jar /path/to/tla2tools.jar -workers 2
    -config <config>.cfg
VrrCoreReincarnation.tla

```

The spellcheck gate for this document is the repository's: `paper/spellcheck.sh paper_results.tex` (Aspell, en_GB).

REFERENCES

- [1] S. Massey, "Diskless Viewstamped Replication with Unbounded Crash-Stop Reincarnation," manuscript, 2026. [Online]. Available: <https://github.com/lua-lunet/uvrr-core>
- [2] S. Massey, "Diskless Viewstamped Replication with Unbounded Crash-Stop Reincarnation — Addendum: proof ladder, weighted reconfiguration and the operator solver," manuscript, 2026. [Online]. Available: <https://github.com/lua-lunet/uvrr-core>
- [3] S. Massey, "uVRR code and proof artifact," 2026. [Online]. Available: <https://github.com/lua-lunet/uvrr-core>